

Piano del Collaudo

Sistema di Gestione e Monitoraggio di Progetti Software

1. Introduzione

Il presente documento descrive il piano di collaudo del sistema di gestione e monitoraggio di progetti software. Vengono definiti i test case per le funzionalità principali — Autenticazione, Gestione Task e Gestione Bug — sia in forma tabellare che tramite codice eseguibile con il framework NUnit (C#).

I test sono organizzati in due livelli:

- Test delle entità del dominio — verificano attributi e metodi delle classi Utente, Task e Bug.
- Test funzionali — verificano i flussi principali del sistema tramite i controller.

2. Autenticazione

I test coprono il flusso di login definito nel diagramma di sequenza Autenticazione: verifica delle credenziali tramite AutenticazioneController, gestione degli scenari di errore e redirect alla home corretta in base al ruolo dell'utente autenticato.

2.1 Test Case — Tabella

ID	Descrizione	Precondizioni	Input	Output Atteso	Esito
TC-A01	Login con credenziali valide (ruolo Sviluppatore)	Utente registrato nel sistema con ruolo Sviluppatore	username: 'mario.rossi', password: 'Password01!'	Autenticazione riuscita, ruolo restituito correttamente, redirect a HomeSviluppatore	
TC-A02	Login con password errata	Utente registrato nel sistema	username: 'mario.rossi', password: 'wrongpass'	Autenticazione fallita, messaggio di errore 'Credenziali errate'	
TC-A03	Login con username inesistente	Nessun utente con tale username registrato	username: 'utente.inesistente', password: 'qualsiasi'	Autenticazione fallita, messaggio di errore 'Credenziali errate'	
TC-A04	Blocco account dopo tentativi ripetuti	Utente registrato, sistema con limite	5 tentativi consecutivi con	Account temporaneamente	

ID	Descrizione	Precondizioni	Input	Output Atteso	Esito
		tentativi attivo	password errata per lo stesso username	bloccato, messaggio di avviso blocco restituito	
TC-A05	Redirect corretto per ruolo Amministratore	Utente con ruolo Amministratore registrato	username e password corretti di un Amministratore	Redirect a HomeAmministratore	
TC-A06	Redirect corretto per ruolo Project Manager	Utente con ruolo Project Manager registrato	username e password corretti di un Project Manager	Redirect a HomeProjectManager	
TC-A07	Redirect corretto per ruolo Tester	Utente con ruolo Tester registrato	username e password corretti di un Tester	Redirect a HomeTester	

2.2 Codice di Test — NUnit (C#)

```
[TestFixture]
public class TestAutenticazione
{
    private Sviluppatore _utente;
    private AutenticazioneController _authController;
    private GestioneUtentiController _utentiController;

    [SetUp]
    public void SetUp()
    {
        _utente = new Sviluppatore("mario.rossi", "Password01!", "Mario",
"Rossi", "mario.rossi@azienda.it");

        _utentiController = new GestioneUtentiController();
        _utentiController.creaUtente(_utente);
        _authController = new AutenticazioneController();
    }

    // TC-A01: credenziali valide
    [Test]
    public void TestLoginValido()
    {
        var esito = _authController.VerificaCredenziali("mario.rossi",
"Password01!")
        Assert.That(esito.Successo, Is.True);
        Assert.That(esito.Ruolo, Is.EqualTo(Ruolo.Sviluppatore));
    }

    // TC-A02: password errata
    [Test]
    public void TestLoginPasswordErrata()
    {
        var esito = _authController.VerificaCredenziali("mario.rossi",
"wrongpass");
        Assert.That(esito.Successo, Is.False);
    }
}
```

```
        Assert.That(esito.Messaggio, Is.EqualTo("Credenziali errate"));
    }

    // TC-A03: username inesistente
    [Test]
    public void TestLoginUtenteInesistente()
    {
        var esito = _authController.VerificaCredenziali("utente.inesistente",
"qualsiasi");
        Assert.That(esito.Successo, Is.False);
        Assert.That(esito.Messaggio, Is.EqualTo("Credenziali errate"));
    }

    // TC-A04: blocco account dopo 5 tentativi
    [Test]
    public void TestBloccoDopoCinqueTentativi()
    {
        for (int i = 0; i < 5; i++)
            _authController.VerificaCredenziali("mario.rossi", "sbagliata");
        var esito = _authController.VerificaCredenziali("mario.rossi",
"Password01!");
        Assert.That(esito.Successo, Is.False);
        Assert.That(esito.AccountBloccato, Is.True);
    }
}
```

3. Gestione Task

I test coprono i flussi definiti nei diagrammi di sequenza Creazione e Assegnazione Task e Aggiornamento Stato Task. Vengono verificati la creazione tramite GestioneTaskPMController, l'aggiornamento di stato tramite GestioneTaskDevController, il metodo isInRitardo() della classe Task e la validazione/rifiuto da parte del ControlloQualitàController.

3.1 Test Case — Tabella

ID	Descrizione	Precondizioni	Input	Output Atteso	Esito
TC-T01	Creazione task con dati validi	Progetto attivo esistente, PM autenticato	nome: 'Implementa login', priorità: Alta, scadenza: data futura, assegnatario: Sviluppatore valido	Task creato in stato Da Fare, storico modifiche registrato	
TC-T02	Creazione task con nome mancante	Progetto attivo esistente, PM autenticato	nome: "", priorità: Alta, scadenza: data futura	Errore di validazione 'Dati non validi', task non salvato	
TC-T03	Creazione task con assegnatario non valido	Progetto attivo esistente, PM autenticato	nome: 'Task test', assegnatario: username inesistente	Errore 'Utente non valido', task non salvato	
TC-T04	Creazione task con dipendenze circolari	Due task A e B esistenti, A dipende da B	Creazione task B con dipendenza da A	Errore 'Dipendenze circolari o task inesistente', salvataggio bloccato	
TC-T05	Aggiornamento stato Da Fare → In Corso	Task in stato Da Fare, assegnato a uno Sviluppatore	taskId valido, nuovoStato: In Corso	Stato aggiornato a In Corso, storico modifiche registrato	
TC-T06	Aggiornamento stato In Corso → Completato	Task in stato In Corso	taskId valido, nuovoStato: Completato	Stato aggiornato a Completato, storico modifiche registrato	
TC-T07	isInRitardo() su task non completato con scadenza superata	Task con scadenza = ieri, stato = In Corso	Chiamata a isInRitardo() sul task	Restituisce true	
TC-T08	isInRitardo() su task completato con scadenza superata	Task con scadenza = ieri, stato = Completato	Chiamata a isInRitardo() sul task	Restituisce false	
TC-T09	Validazione task da parte del tester	Task in stato Completato, Tester autenticato	taskId valido, esito: positivo	Stato aggiornato a Validato, storico verifiche registrato	
TC-T10	Rifiuto task da parte del tester	Task in stato Completato, Tester autenticato	taskId valido, esito: negativo, motivazione: 'Non soddisfa i criteri'	Stato riportato a Da Fare, storico verifiche registrato	

ID	Descrizione	Precondizioni	Input	Output Atteso	Esito
TC-T11	Validazione task non in stato Completato	Task in stato In Corso, Tester autenticato	taskId valido	Errore 'Operazione non consentita', stato invariato	

3.2 Codice di Test — NUnit (C#)

```
[TestFixture]
public class TestTask
{
    private Task _task;
    private GestioneTaskPMController _pmController;
    private Sviluppatore _sviluppatore;
    private GestioneTaskDevController _devController;
    private ControlloQualitaController _qaController;

    [SetUp]
    public void Setup()
    {
        _sviluppatore = new Sviluppatore("dev.user", "Password01!", "Nome",
"Cognome", "email@test.com");
        _task = new Task("Implementa login", "Descrizione del task",
                        Priorita.Alta, DateTime.Now.AddDays(7));
        _pmController = new GestioneTaskPMController();
        _pmController.assegnaTask(_task, _sviluppatore);
        _devController = new GestioneTaskDevController();
        _qaController = new ControlloQualitaController();
    }

    // TC-T01: creazione task valida
    [Test]
    public void TestCreazioneTaskValida()
    {
        Assert.That(_task.Nome, Is.EqualTo("Implementa login"));
        Assert.That(_task.Stato, Is.EqualTo(StatoTask.DaFare));
        Assert.That(_task.Priorita, Is.EqualTo(Priorita.Alta));
    }

    // TC-T05: aggiornamento stato Da Fare → In Corso
    [Test]
    public void TestAggiornaStatoInCorso()
    {
        _devController.AggiornaStato(_task.Id, StatoTask.InCorso);
        Assert.That(_task.Stato, Is.EqualTo(StatoTask.InCorso));
    }

    // TC-T07: isInRitardo() = true
    [Test]
    public void TestIsInRitardoTrue()
    {
        var taskScaduto = new Task("Task scaduto", "",
                                Priorita.Media, DateTime.Now.AddDays(-1));
        taskScaduto.SetStato(StatoTask.InCorso);
    }
}
```

```

        Assert.That(taskScaduto.IsInRitardo(), Is.True);
    }

    // TC-T08: isInRitardo() = false se Completato
    [Test]
    public void TestIsInRitardoFalseSeCompletato()
    {
        var taskScaduto = new Task("Task", "",
                                    Priorita.Bassa, DateTime.Now.AddDays(-1));
        taskScaduto.SetStato(StatoTask.Completato);
        Assert.That(taskScaduto.IsInRitardo(), Is.False);
    }

    // TC-T09: validazione tester
    [Test]
    public void TestValidazioneTask()
    {
        _task.SetStato(StatoTask.Completato);
        var esito = _qaController.validaTask(_task.Id);
        Assert.That(esito.Successo, Is.True);
        Assert.That(_task.Stato, Is.EqualTo(StatoTask.Validato));
    }

    // TC-T10: rifiuto tester
    [Test]
    public void TestRifiutoTask()
    {
        _task.SetStato(StatoTask.Completato);
        var esito = _qaController.RifiutaTask(_task.Id, "Non soddisfa i
criteri");
        Assert.That(esito.Successo, Is.True);
        Assert.That(_task.Stato, Is.EqualTo(StatoTask.DaFare));
    }

    // TC-T11: validazione su task non in stato Completato
    [Test]
    public void TestValidazioneSuTaskNonCompletato()
    {
        _task.SetStato(StatoTask.InCorso);
        var esito = _qaController.ValidaTask(_task.Id);
        Assert.That(esito.Successo, Is.False);
        Assert.That(esito.Messaggio, Is.EqualTo("Operazione non consentita"));
    }
}

```

4. Gestione Bug

I test coprono il flusso definito nel diagramma di sequenza Gestione Bug: segnalazione tramite GestioneBugDevController, assegnazione del responsabile, aggiornamento dello stato fino a Risolto, e verifica finale da parte del GestioneBugTesterController tramite ControlloQualitàController.

4.1 Test Case — Tabella

ID	Descrizione	Precondizioni	Input	Output Atteso	Esito
TC-B01	Segnalazione bug con dati validi	Progetto attivo, Sviluppatore autenticato	nome: 'NullPointerException', descrizione: testo, priorità: Alta, taskAssociato: valido	Bug creato in stato Aperto, storico modifiche registrato	
TC-B02	Segnalazione bug con nome mancante	Progetto attivo, Sviluppatore autenticato	nome: "", priorità: Media	Errore di validazione 'Dati non validi', bug non salvato	
TC-B03	Segnalazione bug con avviso duplicato	Bug con stesso nome già esistente nel progetto	nome e descrizione uguali a bug già esistente	Avviso 'Possibile duplicato rilevato', richiesta conferma	
TC-B04	Assegnazione bug a responsabile valido	Bug in stato Aperto	bugId valido, responsabile: Sviluppatore registrato	Bug assegnato, stato aggiornato a In Lavorazione, storico modifiche registrato	
TC-B05	Assegnazione bug a utente non valido	Bug in stato Aperto	bugId valido, responsabile: username inesistente	Errore 'Utente non valido', assegnazione bloccata	
TC-B06	Aggiornamento stato a Risolto	Bug in stato In Lavorazione	bugId valido, nuovoStato: Risolto	Stato aggiornato a Risolto, storico modifiche registrato	
TC-B07	Transizione di stato non valida	Bug in stato Aperto	bugId valido, nuovoStato: Chiuso (senza passare per Risolto)	Errore 'Transizione non valida', stato invariato	
TC-B08	Chiusura bug da parte del tester (esito positivo)	Bug in stato Risolto, Tester autenticato	bugId valido, esito: positivo	Stato aggiornato a Chiuso, storico verifiche registrato	
TC-B09	Riapertura bug da parte del tester (esito negativo)	Bug in stato Risolto, Tester autenticato	bugId valido, esito: negativo, motivazione: 'Ancora riproducibile'	Stato riportato a Aperto, storico verifiche registrato	
TC-B10	Chiusura bug da parte di utente non autorizzato	Bug in stato Risolto, utente = Sviluppatore	bugId valido, utente non Tester tenta la chiusura	Operazione bloccata, eccezione di accesso non autorizzato	

4.2 Codice di Test — NUnit (C#)

```
[TestFixture]
public class TestBug
{
    private Bug _bug;
    private Sviluppatore _devResponsabile;
    private GestioneBugDevController _devController;
    private GestioneBugTesterController _testerController;
    private ControlloQualitaController _qaController;

    [SetUp]
    public void Setup()
    {
        _bug = new Bug("NullPointerException", "Crash alla login",
Priorita.Alta);
        _devResponsabile = new Sviluppatore("dev.verdi", "Pass123!", "Mario",
"Verdi", "email@test.com");

        _devController = new GestioneBugDevController();
        _testerController = new GestioneBugTesterController();
        _qaController = new ControlloQualitaController();
    }

    // TC-B01: creazione bug valida
    [Test]
    public void TestCreazioneBugValida()
    {
        Assert.That(_bug.Nome, Is.EqualTo("NullPointerException"));
        Assert.That(_bug.Stato, Is.EqualTo(StatoBug.Aperto));
        Assert.That(_bug.Priorita, Is.EqualTo(Priorita.Alta));
    }

    // TC-B04: assegnazione bug
    [Test]
    public void TestAssegnazioneBug()
    {
        var esito = _testerController.assegnaBug(_bug, _devResponsabile);
        Assert.That(esito.Successo, Is.True);
        Assert.That(_bug.Stato, Is.EqualTo(StatoBug.InLavorazione));
    }

    // TC-B06: aggiornamento stato a Risolto
    [Test]
    public void TestRisoluzioneBug()
    {
        _bug.SetStato(StatoBug.InLavorazione);
        var esito = _devController.aggiornaStato(_bug, StatoBug.Risolto);
        Assert.That(esito.Successo, Is.True);
        Assert.That(_bug.Stato, Is.EqualTo(StatoBug.Risolto));
    }

    // TC-B08: chiusura bug da parte del tester
    [Test]
    public void TestChiusuraBugTester()
    {

```

```

        _bug.SetStato(StatoBug.Risolto);
        var esito = _qaController.verificaBug(_bug);
        Assert.That(esito.Successo, Is.True);
        Assert.That(_bug.Stato, Is.EqualTo(StatoBug.Chiuso));
    }

    // TC-B09: riapertura bug da parte del tester
    [Test]
    public void TestRiaperturaBugTester()
    {
        _bug.SetStato(StatoBug.Risolto);
        var esito = _qaController.riapriBug(_bug, "Ancora riproducibile");
        Assert.That(esito.Successo, Is.True);
        Assert.That(_bug.Stato, Is.EqualTo(StatoBug.Aperto));
    }

    // TC-B10: chiusura non autorizzata da Sviluppatore
    [Test]
    public void TestChiusuraNonAutorizzata()
    {
        _bug.SetStato(StatoBug.Risolto);
        var esito = _devController.aggiornaStato(_bug, StatoBug.Chiuso);
        Assert.That(esito.Successo, Is.False);
    }
}

```